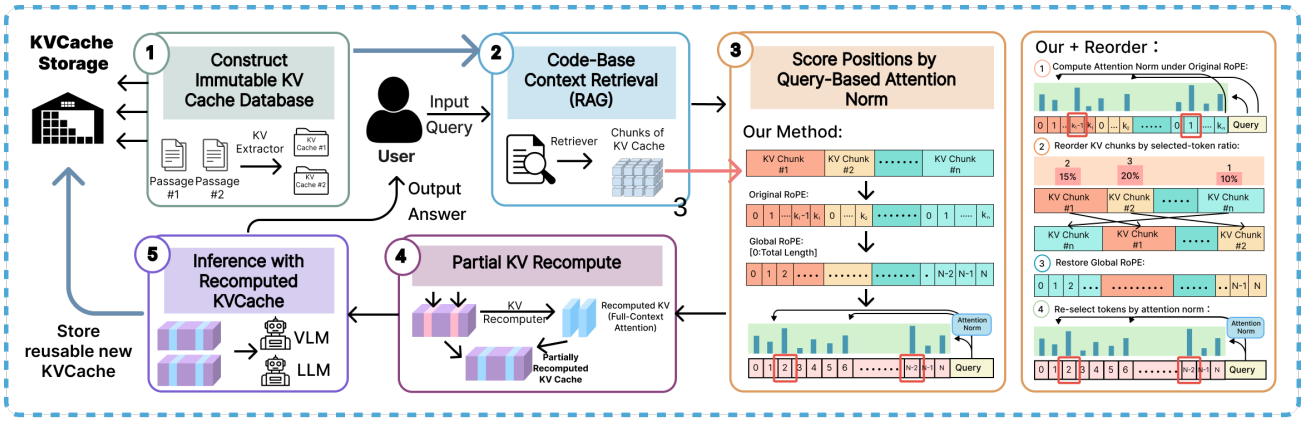


*Figure 1.* Context chunks are prefetched independently using chunk-local RoPE. At inference time, retrieved chunks are concatenated with the prompt, global RoPE positions are reconstructed, and prompt-conditioned attention norms are used to select high-impact tokens for full-context KV recomputation. The recomputed KV states are concatenated with cached chunks, restoring cross-chunk interactions. An optional chunk reordering step places more informative chunks closer to the prompt.

query–token affinity, and effective information flow, by identifying tokens that are structurally positioned to propagate information under the global causal attention induced by retrieval and decoding. Crucially, this property holds only when attention norms are computed under a RoPE ordering consistent with inference-time decoding, using a global positional reconstruction of retrieved chunks; chunk-local or mismatched positional configurations yield unstable and misleading token rankings. Building on this insight, we further propose a retrieved-chunk reordering strategy based on identified information-flow–critical tokens, improving the effectiveness of downstream attention and providing additional validation of our information-flow perspective. Extensive experiments on both LLM and VLM benchmarks show that our approach consistently outperforms existing recomputation methods under comparable budgets.

While our primary focus is retrieval-augmented generation with precomputed document-level KV caches, the proposed chunking-and-recompute strategy also applies when such caches are unavailable. In non-RAG long-context inference, the input can be partitioned into chunks whose KV caches are computed independently, followed by selective recomputation under the global causal mask. This yields two practical benefits: (i) in multi-query or interactive settings, prefetched KV caches can be stored and reused to amortize prefilling cost, and (ii) chunk-wise prefilling naturally enables parallel computation across multiple GPUs, since chunks are processed independently prior to recombination and recomputation.

We summarize our contributions as follows.

- We propose a simple and effective *attention-norm–based* criterion from the query to context tokens that jointly captures semantic relevance and effective information flow under the global causal attention graph.
- We show that reliable recomputation target selection requires an *inference-consistent RoPE ordering*, and introduce a global positional reconstruction of retrieved chunks to enable stable and meaningful token ranking.
- Building on the identified information-flow–critical tokens, we further propose a *retrieved-chunk reordering strategy* that improves downstream attention effectiveness, providing additional validation of our information-flow perspective.
- We evaluate our approach on both LLM and VLM RAG benchmarks, demonstrating consistent improvements over existing selective recomputation methods under comparable efficiency budgets.

## 2. Related Work

### 2.1. KV Eviction & Compression

KV eviction and compression methods reduce long-context inference cost by limiting the size or precision of KV caches during decoding. Eviction-based approaches such as H2O (Zhang et al., 2023) and StreamingLLM (Xiao et al., 2023) discard cached KV states based on heuristics, including recency or attention importance. Compression methods such as Pyramid KV (Cai et al., 2024) and memorization-based approaches (Wu et al., 2022) approximate historical KV representations through hierarchical compression or summary memory. These methods primarily target the memory footprint and access cost during autoregressive decoding. In contrast, our work addresses the prefilling bottleneck under chunk-wise processing, where KV states are computed independently, and cross-chunk interactions

are lost. Rather than shrinking or discarding KV caches, we retain token-level KV states and selectively recompute a small subset of tokens to restore global information flow.

### 2.2. Chunked Input Processing

Chunked input processing has been explored to extend effective context length beyond the model's native window. Context Expansion with Parallel Encoding (CEPE) (Yen et al., 2024) encodes long contexts in parallel using a lightweight encoder and enables a decoder to attend to chunk representations via cross-attention. A key limitation of such approaches is that chunks are encoded independently, and cross-chunk interactions are not explicitly modeled within the chunk representations. Our method instead targets interaction across chunks by selectively recomputing KV states under the full context. Moreover, CEPE requires additional training to align the parallel encoder with the decoder, whereas our approach operates in a plug-and-play manner at inference time without modifying the pretrained model.

### 2.3. KV Recomputation

KV recomputation methods aim to recover accuracy when reusing independently prefetched KV caches by selectively recomputing a small subset of tokens under the full context. CacheBlend identifies tokens whose cached KV states deviate most from full-context attention, while EPIC mitigates positional mismatch by recomputing fixed chunk-initial tokens. These approaches select recomputation targets based on token-level deviation or positional heuristics, without explicitly modeling how information propagates to generated tokens. In contrast, our method selects tokens based on their capacity to transmit information across chunks, jointly accounting for semantic relevance and positional influence under inference-time attention geometry.

## 3. Preliminary

### 3.1. KV Cache

We consider an autoregressive Transformer decoder. Let $\mathbf{h}_t^\ell \in \mathbb{R}^d$ denote the hidden state at layer $\ell$ and position $t$. For each attention head, keys and values are computed as linear projections of the hidden states,

$$\mathbf{k}_t^\ell = \mathbf{W}_K^\ell \mathbf{h}_t^\ell, \qquad \mathbf{v}_t^\ell = \mathbf{W}_V^\ell \mathbf{h}_t^\ell, \tag{1}$$

where the head index is omitted for clarity.

During autoregressive decoding, to avoid recomputing keys and values for previously processed tokens, Transformer decoders maintain a key–value (KV) cache that stores all past $\{\mathbf{k}_s^\ell, \mathbf{v}_s^\ell\}_{s \le t}$ at each layer. We denote the cached tensors up to position $t$ by

$$\mathbf{K}_{\le t}^\ell \in \mathbb{R}^{t \times d_h}, \qquad \mathbf{V}_{\le t}^\ell \in \mathbb{R}^{t \times d_h}, \tag{2}$$

where $d_h$ is the per-head dimension. In practice, inference consists of a *prefilling* phase that computes KV states for an input context of length $L$ in a single forward pass, followed by an iterative *decoding* phase that appends one new KV pair per generated token. In long-context question answering, where the generated answer is short, inference cost is often dominated by the prefilling stage.

### 3.2. Rotary Positional Embedding (RoPE)

Rotary positional embedding (RoPE) encodes positional information by applying position-dependent rotations to queries and keys. For vectors decomposed into 2D subspaces, RoPE applies

$$\mathrm{RoPE}(\mathbf{x}, t) = \begin{bmatrix} \cos(\theta_i t) & -\sin(\theta_i t) \\ \sin(\theta_i t) & \cos(\theta_i t) \end{bmatrix} \mathbf{x}, \tag{3}$$

where the angular frequencies are defined as

$$\theta_i = 10000^{-2i/d}, \quad i = 0, \ldots, \frac{d}{2} - 1. \tag{4}$$

Lower-index dimensions correspond to higher frequencies and are more sensitive to positional differences, while higher-index dimensions encode lower-frequency components. Consequently, the effectiveness of attention interactions depends on the absolute positions at which tokens are placed, which becomes particularly relevant when KV states are computed independently and later reused or recomputed under different positional assignments.

## 4. Method

In this section, we introduce our method for selecting recomputation targets by jointly considering token semantic relevance and positional influence, with the goal of facilitating information propagation during decoding.

### 4.1. Input Chunking and Prefilling

Let the input consist of $N$ tokens, which we partition into $K$ disjoint chunks $\{C_1, \ldots, C_K\}$. Each chunk serves as a basic unit for prefilling and can correspond to a naturally independent segment (e.g., a document or an image), or a contiguous partition of a single long input (e.g., a text sequence or a visual token grid).

During chunk-wise prefilling, each chunk is processed independently to compute its key–value (KV) cache. Within a chunk $C_i$, positional indices are assigned locally: tokens are indexed from 0 to $|C_i| - 1$ according to their order inside the chunk, regardless of their absolute position in the full input. Keys and queries are then computed using RoPE based on these chunk-local positions, and the resulting KV states are stored in the cache associated with $C_i$. As a result, chunk-wise prefilling encodes KV states under chunk-local

which frequency placement and prompt–context proximity can arise in practice.

### 4.3. KV reorder

Beyond KV prefilling and token selection, we further consider the ordering of chunks at inference time as an additional degree of freedom for facilitating information flow. Under RoPE-based causal decoding, tokens placed closer to the prompt in the sequence are more likely to interact with prompt queries through attention, which facilitates more effective propagation of contextual information to downstream generated tokens. We therefore introduce an optional chunk reordering step for settings where input chunks correspond to **independent segments**, such as multi-document retrieval, and where no intrinsic sequential order needs to be preserved. For inputs with intrinsic sequential structure, such as a single long document, reordering may disrupt coherence and is therefore not used. To enable chunk reordering, we first derive chunk-level importance scores.

Concretely, we perform an initial round of token selection within each chunk independently, using local RoPE rather than global positions. This makes token selection within each chunk comparable and prevents chunks closer to the prompt from being systematically favored. After reordering, we reselect tokens under the updated chunk order to obtain the final set of recomputation tokens. This second selection pass accounts for the fact that reordering alters the original prompt–context RoPE geometry, and ensures the selected tokens remain important under the new layout. Autoregressive decoding is then performed using the reordered chunks and the corresponding recomputed KV states.

Thus, chunk reordering leverages the freedom in chunk ordering to further align RoPE geometry with information flow in the decoding phase, complementing token selection under chunk-wise prefilling. More details on KV recomputation implementation can be found at Appendix. B

## 5. Ablation Result

### 5.1. RoPE Geometry

We ablate different RoPE geometry configurations to study how positional allocation during token selection affects downstream performance.

Table 1 shows that RoPE geometry has a substantial impact on recomputation effectiveness. Among the tested configurations, GLOBAL consistently achieves the best performance across all four benchmarks. In contrast, configurations that assign prompt and context tokens to separate or truncated positional ranges (e.g., HL–HP and TL–TP) result in noticeably lower scores. This suggests that mismatches between the positional layout used for selection and the actual decoding order can degrade the quality of selected recomputation targets.

These results indicate that effective selective recomputation benefits from evaluating token importance under a positional layout that closely matches inference-time decoding. Based on this observation, we adopt GLOBAL as the default RoPE geometry for token selection in our method. In the reordering setting, we use HL–TP for the first-stage selection to encourage diversity across chunks, followed by GLOBAL for the second-stage selection to refine recomputation targets under the final positional layout.

*Table 1.* Ablation of RoPE geometry configurations on Qwen under the passage-split setting. Higher is better.

| Method | 2WikiMQA | MuSiQue | HotpotQA | NarrativeQA |
|---|---|---|---|---|
| HL–HP | 0.4455 | 0.2871 | 0.5529 | 0.1481 |
| TL–TP | 0.4458 | 0.2970 | 0.5693 | 0.1923 |
| HL–TP | 0.4722 | 0.3072 | 0.5651 | 0.2106 |
| GLOBAL | **0.5019** | **0.3386** | **0.5954** | **0.2288** |

### 5.2. RoPE Similarity

To further understand why our selection strategy facilitates effective information propagation, we analyze the RoPE similarity between prompt tokens and the selected context tokens. Importantly, this analysis blocks the contribution of token semantics: similarities are computed purely from the RoPE embedding matrices. Specifically, we measure the similarity between the RoPE embeddings of prompt tokens and those of the selected context tokens, and report both the Mean-of-Max (MoM) similarity and the maximum similarity across selected tokens. Higher values indicate that selected tokens occupy RoPE frequency bands that are more closely aligned with the prompt, and are therefore more positionally reachable under causal decoding.

| | | 2WikiMQA | | HotpotQA | |
|---|---|---|---|---|---|
| Model | Method | MoM | Max | MoM | Max |
| *LLaMA* | Norm-based | **0.5324** | **0.9773** | **0.5219** | **0.9766** |
| | CacheBlend | 0.5243 | 0.9133 | 0.5191 | 0.8570 |
| | EPIC | 0.5049 | 0.6734 | 0.4985 | 0.6852 |
| *Qwen* | Norm-based | **0.4548** | **0.9805** | 0.4179 | **0.9805** |
| | CacheBlend | 0.4505 | 0.8078 | **0.4226** | 0.8891 |
| | EPIC | 0.4129 | 0.5656 | 0.3835 | 0.6555 |

*Table 2.* RoPE similarity statistics on long-context QA benchmarks. We report Mean-of-Max (MoM) and Max scores across different inference strategies. Higher is better.

As shown in Table 2, norm-based selection consistently yields higher RoPE similarity scores than prior methods across both datasets and model families, under both the Mean-of-Max and Max metrics. This result reveals an alignment between attention salience and RoPE-induced